

Hash Maps

KEY-VALUE PAIR ASSOCIATIONS

Data Structures: Module 23

codeHive

1. The Power of Association

A **Hash Map** (or Dictionary) maps unique **Keys** to specific **Values**. It is the ultimate tool for looking up detailed data using a simple identifier.

Key | 123-4567



Value | Charlotte

Essential Vocabulary:

- **Entry:** The complete Key-Value pair stored in the map.
- **Key:** The unique ID (must be unique to avoid overwriting data).
- **Value:** The person's name, profile, or any data attached to the key.

2. Numerical Indexing (The Logic)

How does "123-4567" find "Charlotte"? The process uses digit summation and modulo arithmetic.

Hash Process for Key "123-4567"

$$1+2+3+4+5+6+7 = 28$$

$$28 \% 10 = \text{Index } 8$$

Direct Access Advantage: By jumping directly to Index 8, we bypass searching every other entry. Search time is **$O(1)$** on average.

codeHive

3. Implementation

The Python SimpleHashMap class handles entry updates (if the key exists) and chaining (if a collision occurs).

```
class SimpleHashMap:
    def __init__(self, size=10):
        self.buckets = [[] for _ in range(size)]

    def hash_function(self, key):
        # Sum digits only
        numeric_sum = sum(int(c) for c in key if c.isdigit())
        return numeric_sum % 10

    def put(self, key, value):
        idx = self.hash_function(key)
        bucket = self.buckets[idx]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value) # Update
                return
        bucket.append((key, value)) # Add new

    def get(self, key):
        idx = self.hash_function(key)
        for k, v in self.buckets[idx]:
            if k == key: return v
        return None
```

4. Efficiency Verdict

Search: $O(1)$

Insert: $O(1)$

Worst Case: $O(n)$

Note on Memory: Using a Social Security Number directly as an array index would require an array with 100 billion slots—a massive waste. Hash Maps map these huge keys into a small, manageable number of buckets.

© 2026 codeHive Academy • Relational Data Structures